9b. Stack vs Heap

Martin Alfaro
PhD in Economics

INTRODUCTION

Memory allocation occurs whenever a new object is created, requiring the system to set aside a portion of Random Access Memory (RAM) to store the object's data. For simple objects, the compiler may actually optimize away the allocation entirely and directly keep their data in CPU registers.

When RAM is required, the system manages it using two distinct logical models: the stack and the heap. These areas aren't physical locations, but rather management models that govern how memory is handled. Typically, the specific storage location is determined by the nature of the object. For instance, vectors in Julia are stored on the heap, whereas tuples can be allocated either on the stack or optimized into CPU registers.

The choice between heap and non-heap allocations has significant implications for performance. Allocating memory on the heap is comparatively an expensive operation. It requires a systematic search for an available block of memory, bookkeeping to track its status, and an eventual deallocation process to reclaim the space once it's no longer needed. [1] Stack, by contrast, is simpler and therefore faster.

The performance gap between stack and heap allocations can easily become a critical bottleneck when an operation is performed repeatedly. This disparity in performance explains a widespread convention in programming (including Julia), where **memory allocations exclusively refer to heap allocations**. In the following, we provide a brief overview of how each operates.

STACK ALLOCATIONS

The stack is typically reserved for storing data whose size and lifetime can be determined at compile time. This includes many primitive values (e.g., integers, floating-point numbers, and characters), as well as some fixed-size immutable aggregates like tuples. These objects have a known fixed layout, which allows the compiler to place them on the stack or even eliminate the allocation entirely by keeping their contents in CPU registers. Their fixed size and predictable lifetime make allocation and deallocation extremely efficient.

The primary downside of the stack is its limited capacity, which makes them suitable only for objects with a few elements. Indeed, attempting to allocate more memory than the stack can accommodate will result in a "stack overflow" error, causing program termination. And, even if an object fits on the stack, allocating too many elements will significantly degrade performance. [2]

HEAP ALLOCATIONS

Common objects stored on the heap include arrays—such as vectors and matrices—as well as strings. The heap is designed to support data structures whose size can't be determined at compile time, with the possibility of growing and shrinking dynamically. These objects can be arbitrarily large, up to the limits of available RAM, and often have lifetimes that extend beyond a single function call. As a result, they must be managed in a more flexible region of memory than the stack can provide.

While the heap offers greater flexibility, its more complex memory management comes at the cost of slower performance. [3] Due to its overhead, the upcoming sections of this book will discuss strategies for reducing heap allocations. These include computational techniques that translate vectorized operations into repeated scalar operations, as well as programming practices that favor mutating existing objects over the creation of new ones.

FOOTNOTES

[1.] This deallocation process is automated in Julia by what's known as a garbage collector.

[2.] There's no hard and fast rule about how many elements are "too many". Benchmarking is the only reliable way to determine the performance consequences for each particular case. As a rough guideline, objects with more than 100 elements will certainly suffer from poor performance, while those with fewer than 15 elements will benefit from stack allocation.

[3.] Heap memory is managed by what's known as the *garbage collector*, which automatically identifies and reclaims memory no longer in use. This process, while convenient, can be computationally expensive.